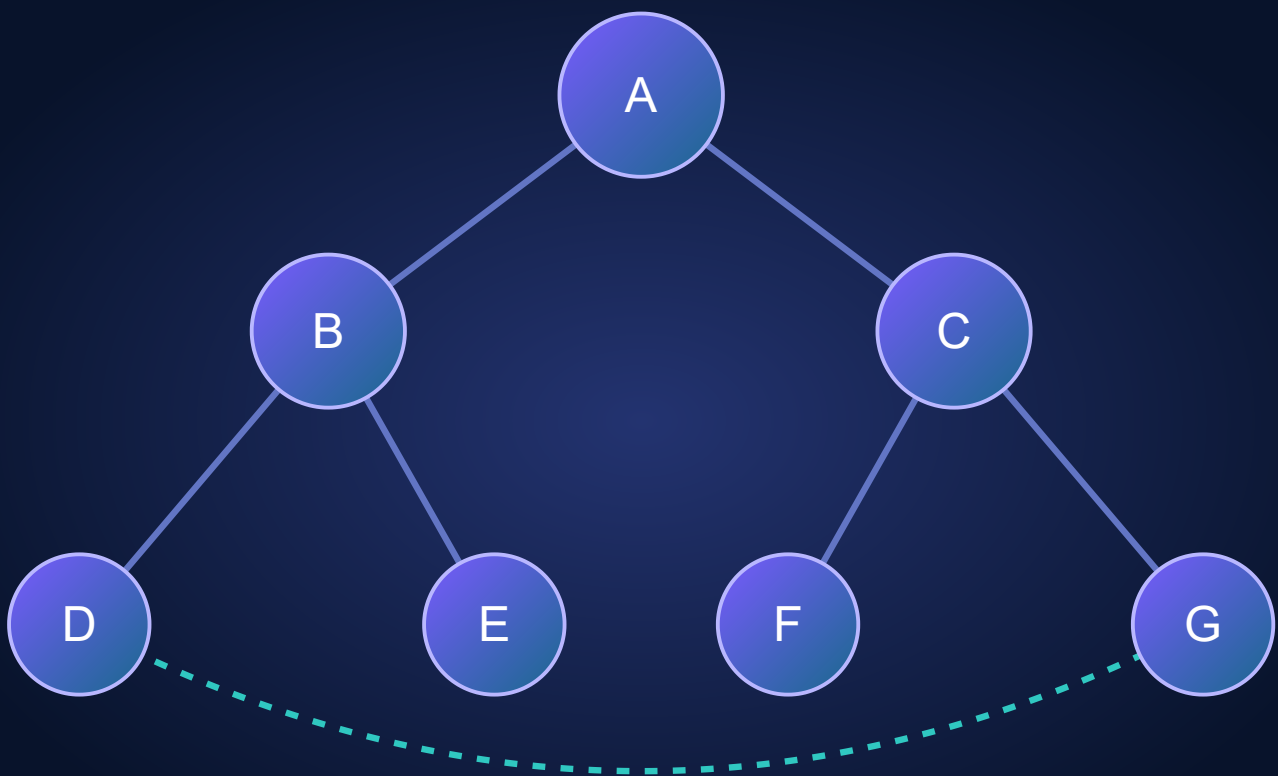


PROGRAMMING · INTERMEDIATE

Data Structures & Algorithms

See the algorithm. Understand every step.



TRACE · REASON · COMPARE

A visual, practice-first path through data structures, algorithms and interview-grade problem solving.

 12-16 weeks 5 modules 20 lessons[Explore the course →](#)

Concepts, worked examples, assignments and a coherent course project.

WHO IT IS FOR · WHAT YOU WILL LEARN

Your next capability.

Learn by manipulating live arrays, linked nodes, trees and graphs; step through algorithms; inspect state and complexity; and solve progressively harder coding challenges.



Choose the right data structure for a constraint



Trace and implement common algorithms



Explain time and space complexity



Solve interview-grade problems with tested code



Audience

- C and C++ learners
- Software engineering students
- Developers preparing for technical interviews



Prerequisites

- Programming fundamentals in C, C++, Java, Python or JavaScript

FROM CONCEPT TO EVIDENCE



One curriculum, three connected views

This guide summarizes the same modules and lessons shown on the course overview and in the learning workspace. The next pages explain the scope and practical evidence for every module.

DETAILED MODULE CURRICULUM · 1-3

Build the foundations.

The module names, lesson sequence and evidence below match the course learning workspace.

1 Complexity, Arrays and Strings

Compare algorithms using input size, operation counts and explicit assumptions.

1.1 Big O with live traces

Big O describes how a resource bound grows with input size; it is not an exact stopwatch prediction.

1.2 Array memory visualizer

An array supports indexed access because elements occupy a predictable contiguous layout.

1.3 Two pointers and sliding windows

Two pointers and sliding windows exploit structure to avoid repeating work.

1.4 Searching and sorting

Binary search needs an ordered search space and consistent boundary updates.

You will produce: Annotated trace, invariant and complexity comparison.

2 Linked Structures

Choose a linked structure based on operations rather than its visual appearance.

2.1 Singly linked list simulation

A singly linked node stores a value and a next reference.

2.2 Doubly linked lists

A doubly linked node also records its predecessor.

2.3 Stacks and queues

A stack removes the most recently added item; a queue removes the oldest.

2.4 Hash tables

A hash table maps keys to positions through a hash function and collision strategy.

You will produce: Linked-structure traces and edge-case tests.

3 Trees and Heaps

Use tree structure to organize search and priority operations.

3.1 Binary tree explorer

A binary tree allows at most two children per node.

3.2 BST operations

A binary search tree maintains an ordering rule between each node and its subtrees.

3.3 DFS and BFS

Depth-first traversal explores a branch before returning; breadth-first traversal uses a queue to visit levels.

3.4 Heaps and priority queues

A heap maintains a priority relationship, not a fully sorted sequence.

You will produce: Tree explorer trace and invariant tests.

DETAILED MODULE CURRICULUM · 4-5

Apply, integrate and demonstrate.

The module names, lesson sequence and evidence below match the course learning workspace.

4 Graphs and Dynamic Programming

Solve graph problems by matching the algorithm to the edge model.

4.1 Graph builder

Represent vertices and edges explicitly, including whether edges are directed or weighted.

4.2 BFS and DFS simulation

BFS uses a queue and discovers vertices by edge distance in an unweighted graph.

4.3 Shortest paths

BFS minimizes edge count only when that matches cost.

4.4 Dynamic programming patterns

Dynamic programming stores solutions to recurring subproblems.

You will produce: Graph trace, path reconstruction and assumptions.

5 Problem Solving and Capstone

Explain a correct solution before optimizing or presenting it.

5.1 Pattern selection

Choose a pattern by recognizing an invariant or repeated subproblem, not by matching a keyword.

5.2 Constraint-driven design

Constraints determine feasible time and memory.

5.3 Timed problem sets

Timed practice should include reading, planning, implementation and validation.

5.4 Algorithm visualizer capstone

Build a visualizer that distinguishes current state, discovered state and queued work.

You will produce: Algorithm visualizer, tests and complexity explanation.

Practice is part of every module

Work through the example, record your reasoning, complete the module assignment and compare your work with the review criteria.

PROJECTS AND WORKING METHODS

Visual algorithm explorer

Trace data structures, compare approaches and build a small visual explanation of an algorithm with test cases and complexity analysis.



Step-by-step graph or tree trace



Correctness and edge-case tests



Time and space complexity explanation



Tools and methods

Interactive SVG simulations

Code editor and test runner

Complexity visualizer

Practice judge

Tools are learning choices, not partner endorsements.

Inside module 1: a worked example

Sorted array: [2, 4, 6, 8, 10]

Search 8: low=0, high=4, mid=2 -> value 6

Move low=3; mid=3 -> value 8

Each step preserves: if the target exists, it lies in [low, high].

The invariant explains correctness; the shrinking interval explains termination and logarithmic comparisons.

How your work is reviewed

Show the artifact, explain your decisions, test the stated assumptions and identify limitations. Module assignments build toward the course project rather than replacing it with a single example.

DELIVERY, COMPLETION AND NEXT STEPS

Know what you are signing up for.

**Delivery**

Self-paced online · 12–16 weeks

Confirm current schedule, cohort arrangements and enrollment terms with the academy.

**Completion requirements**

Complete the required coursework and module assignments, and pass the course assessment. A course completion certificate is available after those requirements are met.

What the learning workspace includes**Follow the modules**

Expand the curriculum and open the exact lesson you need.

**Apply the concepts**

Read explanations, inspect examples and complete module practice.

**Keep your evidence**

Use resources, notes, discussion and assignments to support your learning.

Learning commitments, not outcome guarantees

Completion is not a placement, employment or performance guarantee. The course project is educational evidence; any production use requires appropriate review and validation.

Start with the curriculum. Continue with the first lesson.

Review your starting point, download this guide and explore the connected course experience.

[Go to the first lesson](#) →

Client-review edition. The static learning workspace saves demonstration progress locally; production enrollment, assessment, communications and certificate issuance are not connected.